

XPCE and Prolog

Manuel Hernandez

January 22, 2009

Abstract

Declarative Programming includes at least two paradigms: the Functional and the Logic one. Both paradigms provide interaction mechanisms between programs and users. Interaction is a side-effect. In Functional Programming side-effects have been successfully treated by using Monad Theory, but in Logic Programming there is not exist an analogous mental tool.

XPCE is a modern graphical environment to deal some topics related to input/output. XPCE is distributed along with Swi-Prolog; XPCE is pragmatically coupled to Prolog. To integrate XPCE and Prolog following a declarative semantics I follow some game-based concepts. Users have a specific and disciplined role within an extended (timed) least Herbrand model, where queries and facts live armoniously.

Chapter 1

Pragmatics: Fun begins

XPCE is a graphical environment based on the philosophy of Object Oriented Programming (OOP) (classes, methods, inheritance, messages). XPCE is distributed along with Swi-Prolog. XPCE and Prolog live together within a same system, in spite of having distinct theoretical basis. In the following, we illustrate some isolated aspects of XPCE, and the relationship between XPCE and SWI-Prolog.

1.1 Graphical examples

We begin with a typical example, the “Hello World” program:

XPCE program

```
1 :-use_module(library(pce)).  
2 main :-  
3     new(P,window('Hello!')),  
4     send(P,display,text('Hello world!'),point(20,20)),  
5     send(P,open).
```

The result of this program is shown on Fig. 1.1. In line 3 the window is bound to a variable (P, in this case). The variable is used to name the window. A message can be sent to this window by using the variable. Variables are local within clauses; if we want to send a message to an object used by several clauses, the best is to give to the object a specific name. Using local variables is a good on-line practice in some parts of a programs. In case of working



Figure 1.1: A graphical window in XPCE.

with few objects, or when we have to share global information within the whole program (across several clauses), the best practice is to globally name objects. We will see how to name an object later.

On Linux, we write `> pl -s xpce_example01.pl -g main`, and we obtain the following output:

The command `new` allows us to define new objects. Before to get a best landscape of `new` and friends, we give a brief information about some valuable resources to get a better understanding of XPCE.

Although we continue with some basic notes about XPCE, SWI-Prolog, via XPCE itself, includes a very complete manual explaining every class within XPCE, and the associated methods. Thus, a good comprehension of how to use the manual is necessary, particularly when we develop not trivial applications.

XPCE has also a class hierarchy. There is tool named 'Class Hierarchy'. This tool gives a graphical landscape of every class built-in in XPCE. Moreover, there is a tool called 'Class Browser'. Almost everything related to XPCE is here. A search tool is also provided. To learn to use this tool the best is to use it. Finally, two extra notes should be ready right now: first, in XPCE there are many references to objects globally available (`pce`, `prolog.x`, to say). Use the Class Browser to discover this kind of objects. The following note is about the XPCE/Prolog predicates. These predicates allow to communicate XPCE and Prolog, perhaps in a bidirectional way. By

using these predicates, XPCE and Prolog are interconnected. Later, we will say something more about these predicates.

There are some useful commands to get extra help: `help(identifier)`, for example, gives the documentation associated to *identifier*. Similarly, `show_slots(object)` shows the optional parts of a named object.

In general, `manpce` is a command to activate helping resources. For example, we can learn many concepts and techniques of XPCE from the given examples.

Having given these isolated notes about helping in XPCE, we continue with the following four basic predicates or command of XPCE.

There are four basic commands to work within XPCE:

1. `new/2`;
2. `get/[3-13]`;
3. `send/[2-12]`; and,
4. `free/1`.

The `new/2` command creates an object, belonging to certain class, and this object is named *identifier*.

The `get/3` command obtains information of the object named *identifier*. When the state of the object changes, `get` updates the old information, and we can get the new state of the object.

The `send/2` command sends information to an object. This information will modify certain characteristics of the object.

Finally, the `free/1` command, when applied to an object, will destroy the object. Consequently, the resources used by the object will be liberated.

An object belongs to a *class*. A class consists of *objects* and *methods*. Each class has associated methods to be applied to its objects. For example, in the following program we have attached a size to our window:

```
_____ XPCE program _____  
1 :-use_module(library(pce)).  
  
2 main :-  
3     new(P,window('Hello!',size(200,100))),  
4     send(P,display,text('Hello world!'),point(20,20)),  
5     send(P,open).  
_____
```



Figure 1.2: A sized window and a modified font.

With the next example, we illustrate how to change some properties of a piece of text:

XPCE program

```

1 :-use_module(library(pce)).

2 main :-
3     new(P,window('Hello!',size(200,100))),
4     send(P,display,text('Hello world!',
5                           center,font(times,bold,18)),point(20,20)),
6     send(P,open).
```

The Fig.1.2 shows the resulting window.

Moreover, we can associate to the text a *recogniser*. This recogniser allows to hold on the text, by pressing the *left-mouse botton* and the *control key*:

XPCE program

```

1 :-use_module(library(pce)).

2 main :-
3     new(P,window('Hello!',size(200,100))),
4     send(P,display,
5           new(Text,text('Hello world!',
6                           center,font(times,bold,18))),point(20,20)),
```

```
7      send(Text,recogniser,new(move_gesture(left,c))),
8      send(P,open).
```

In the line 7, `left` refers to the left-button of the mouse. The options are: `left`, `middle`, and `right`. Similarly, `c` refers to the control-key. Other options are: `s`, for the shift-key, and `m`, for the meta-key (see `help(modifier)` within Prolog). Several recognisers can be in the play.

To get some familiarity with recognisers, we have the following example:

```
                                XPCE program
1 :-use_module(library(pce)).

2 main :-
3     new(@pict,picture('Graphics in XPCE')),
4     send(@pict,display,new(@box,box(100,100))),
5     send(@box,recogniser,new(move_gesture)),
6     send(@pict,open).
7 :-main.
```

(In line 7 `:-main.` allows us to begin immediately the program, without writing the `-g main` part on the prompt.)

In this example, we open an enviroment where to place our graphical objects. A *picture* is like a blackboard: we put, move, draw, and modify graphical objects on this blackboard. We can send a graphical object to this picture. In the previous code, we have drawn a box. Also, we have associated to this box a recogniser. This recogniser acts under the even of holding the middle of a mouse pressed. When the pointer of the mouse is moving, the box is also dragged to a new position.

In the following program we use several recognisers. We also illustrate the important concept of how to call a Prolog predicate from XPCE.

```
                                XPCE program
1 :-use_module(library(pce)).

2 main :-
3     new(P>window('Hello!',size(200,100))),
4     send(P,display,
```



Figure 1.3: Placing text.

```

5         new(Text,text('Hello world!',
6             center,font(times,bold,18)),point(20,20)),
7     send(Text,recogniser,new(move_gesture(left,c))),
8     send(P, recogniser,
9         click_gesture(left,'',single,
10             message(@prolog,new_text,
11                 @receiver,@event?position))),
12     send(P,open).

13 new_text(P,Pos) :-
14     new(NText,
15         text('Bye world!',center,
16             font(helvetica,italic,20))),
17     send(P,display,NText,Pos).

```

The result of this program is shown in Fig. 1.3. The function `@event?position` is a method of the class `point(int,int)`.

By only changing some parameters, we can obtain a new program dealing with circles, instead of text:

XPCE program

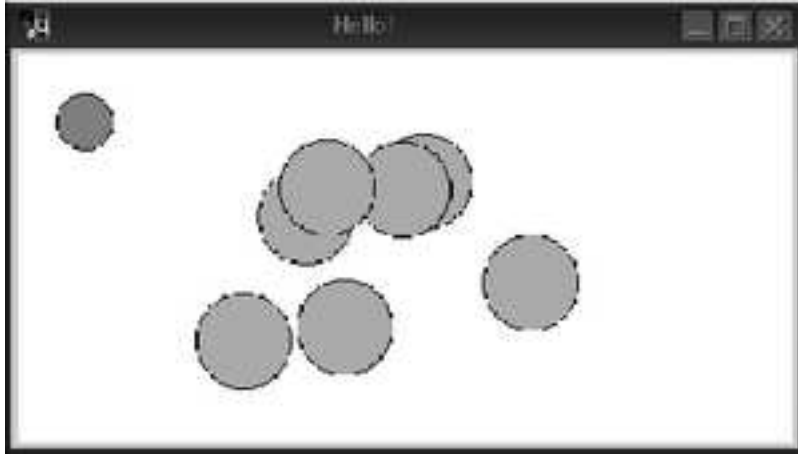


Figure 1.4: Placing circles.

```

1 :-use_module(library(pce)).

2 main :-
3     new(P,window('Hello!',size(200,100))),
4     send(P,display,
5         new(Circle,circle(30)),point(20,20)),
6     send(Circle,fill_pattern,colour(green)),
7     send(Circle,recogniser,new(move_gesture(left,c))),
8     send(P, recogniser,
9         click_gesture(left, '', single,
10             message(@prolog, new_circle,
11                 @receiver, @event?position))),
12     send(P,open).

13 new_circle(P,Pos) :-
14     new(NCircle,circle(50)),
15     send(NCircle,fill_pattern,colour(orange)),
16     send(P,display,NCircle,Pos).

```

Now, the result of this program is shown in Fig. 1.4.

If we want to put circles only when the pointer of the mouse is over certain area over the picture, we have to implement something like the following program:

```
1 :-use_module(library(pce)).

2 main :-
3     new(P,window('Hello!',size(200,100))),
4     send(P,display,
5         new(Circle,circle(30)),point(20,20)),
6     send(Circle,fill_pattern,colour(green)),
7     send(Circle,recogniser,new(move_gesture(left,c))),
8     send(P, recogniser,
9         click_gesture(left, '', single,
10                        message(@prolog, new_circle,
11                                   @receiver, @event?position))),
12     send(P,open).

13 new_circle(P,Pos) :-
14     new(NCircle,circle(50)),
15     get(Pos,x,X),
16     get(Pos,y,Y),
17     rect(X,Y),
18     send(NCircle,fill_pattern,colour(orange)),
19     send(P,display,NCircle,Pos).
```

It is also possible to modify the place where the circle is displayed:

```
new_circle(P,Pos) :-
    new(NCircle,circle(50)),
    get(Pos,x,X),
    get(Pos,y,Y),
    rect(X,Y),
    send(Pos,x,100),
    send(NCircle,fill_pattern,colour(orange)),
    send(P,display,NCircle,Pos).
```

Now, we introduce the important concept of path. A path is a sequence of points. Graphically, a path is a curve passing over the sequence of points.

As an example, we draw the sine curve:

```
XPCE program
1 :-use_module(library(pce)).

2 draw_sine :-
3     send(new(Pict, picture), open),
4     send(Pict, display, new(P, path)),
5     (    between(0, 360, X),
6         Y is sin((X * 6.283185)/360) * 100+100,
7         send(P, append, point(X, Y)),
8         fail
9     );   true
10    ).
```

We can use a path to define new shapes. Moreover, we can give a colour to our shape.

```
XPCE program
1 :-use_module(library(pce)).

2 main :-
3     new(@pict,picture('Hi!')),
4     new(@path,path),
5     send(@path,points,
6         point(10,10),
7         point(100,20),
8         point(30,50),
9         point(20,50)
10        ]),
11     send(@path,closed,@on),
12     send(@path,fill_pattern,colour(yellow)),
13     send(@pict,display,@path),

14     new(@path2,path),
15     send(@path2,points,[
```

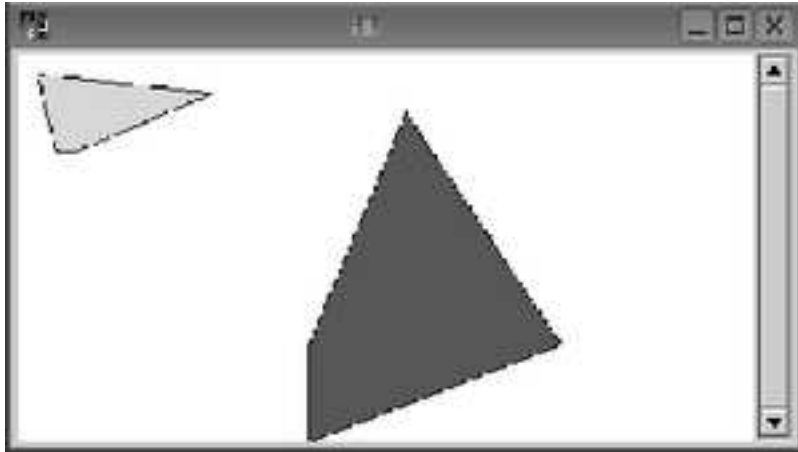


Figure 1.5: Designing some shapes.

```

16         point(150,150),
17         point(150,200),
18         point(280,150),
19         point(200,30)
20     ]),
21     send(@path2,closed,@on),
22     send(@path2,fill_pattern,colour(red)),
23     send(@pict,display,@path2),
24
25     send(@pict,open).

```

Thus, there is not limit in the number of shapes we can design. Triangles, paralepipeds, and so, can easily be drawn modelled in XPCE.

In the following example we collect several objects under a common name (`@block`) through a *device*. When we associate a *move_gesture*, all the elements of the device are treated as only one object.

```

XPCE program
1 :-use_module(library(pce)).

2 main :-

```

```

3      new(@pict,picture('Graphics in XPCE')),
4      new(@block,device),
5      send(@block,display,circle(20),point(20,20)),
6      send(@block,display,circle(20),point(0,20)),
7      send(@block,display,circle(20),point(20,0)),
8      send(@block,display,circle(20),point(0,0)),
9      send(@pict,display,@block,point(100,100)),
10     send(@block,recogniser,new(move_gesture)),
11     send(@pict,open).

12 :-main.

```

Devices is a powerful tool statically link several objects. A kind of dynamic link can also be built to link several objects, by using lines to connect them. We will deal with this theme later.

Images is a fundamental tool within the modern trends of multimedia. In XPCE we can see and manipulate images, as the following program shows:

```

XPCE program
1 :-use_module(library(pce)).

2 main :-
3     new(P,picture('Hola')),
4     new(I,image('firenze.jpg')),
5     new(B,bitmap(I)),
6     send(P,display,B),
7     send(P,open).

```

The output is displayed in Fig. 1.6.

There are some tools within XPCE to change an image.

Finally, we give an introduction to *dialogs* by giving a practical use of a dialog together a picture environment. To add extra controls to a picture, the best is deal with a new dialog, outside to the picture:

```

XPCE program
1 :-use_module(library(pce)).
2 main :-

```



Figure 1.6: A graphical window in XPCE.

```

3      new(@pict,picture('Graphics in XPCE')),
4      send(@pict,display,new(@box,box(100,100))),
5      send(@pict,display,new(@circle,circle(100))),
6      send(@circle,recogniser,new(move_gesture)),
7      send(@box,recogniser,new(move_gesture)),
8      send(@pict,open),
9      wdialog.
10 wdialog :- new(@diag,dialog('Controls')),
11           send(@diag,append, button(quit,message(@prolog,halt))),
12           send(@diag,open).

13 :-main.

```

Enough to be an introduction to the graphical possibilities of XPCE. In the next section, we talk at some extent about dialogs.

1.2 Dialog examples

This is a minimal version of a dialog:

XPCE program

```

1 :-use_module(library(pce)).

2 main :-
3     new(D,dialog('Hello!')),
4     send(D,open).

```

In the next example, we exemplify how to append a button to the initial dialog:

XPCE program

```

1 :-use_module(library(pce)).

2 main :-
3     new(D,dialog('Hello!')),
4     send(D,append,button(quit,message(@prolog,halt))),
5     send(D,open).

```

To use XPCE, we need to include within our source file the following line:

```
:-use_module(library(pce)).
```

(Hence, the programmer can consider XPCE as a library, by the way.)
Next, a file can have the following lines:

XPCE program

```

1 main :-
2     new(@testdialog,dialog('My first window')),
3     send(@testdialog,open).

4 :-main.

```

The predicate *main* is only a possible name to start our program (*main* is not a reserved word in Prolog).

The part `send(testdialog,open)` activates the dialog named `testdialog`.

It is possible to directly use the prompt of SWI-Prolog to do sequentially our window:



Figure 1.7: The quit button in a XPCE dialog.

```
?- new(@testdialog,dialog('My first window')).
```

Yes

```
?- send(@testdialog,open).
```

Yes

In the previous use of button we have not associated an event to this button. In the following example we associate the event of (totally, from XPCE and Prolog) exiting when the button Exit is pressed.

We use a file to give our examples thinking in programs having many lines of code.

```
_____ XPCE program _____  
1 main :-  
2     new(@testdialog,dialog('My first window')),  
3     send(@testdialog,append,  
4         button('Exit',message(@prolog,halt))),  
5     send(@testdialog,open).
```

We have added new concepts here. We can associate with the button exit an event. The associated event is taken directly from Prolog: `message(prolog,halt)` executes, from Prolog, the (built-in) predicate `halt`. Other predicates written in Prolog can be called in this way.

This adds a button, labeled Exit, to our dialog. Buttons are standard components of interaction between human beings and computers. There are other components commonly found in many modern window-system managers: menus, radio-buttons, sliders, lists, for example.

By pressing the alt-key and 'q' at the same time, or by pressing the button Quit with the pointer of the mouse, we exit from this dialog.

Other controls can be incorporated to a dialog window: `text_item`, `int_item`, `slider`, `menu`, `menu_bar`, and `label`.

Now we illustrate the use of `get`:

```

                                XPCE program
1 :-use_module(library(pce)).

2 main :-
3     new(@testdialog,dialog('My first window')),
4     send(@testdialog,append(text_item(name))),
5     send(@testdialog,append(button(exit,message(@prolog,halt)))),
6     get(@testdialog,member(name),Text),
7     get(Text,selection,NText),
8     send(@testdialog,open).

9 :-main.

10 q :- free(@testdialog).
```

The objects created by `new` are global. This is a very important departing from the practice of programming of Prolog, because in Prolog variables are local to the clauses where they appear. In fact, there is not exist, as far as we know, a suitable theory integrating something like input-output and Logic Programming: this is called the input-output problem of Logic Programming. Hence, on the one hand, XPCE and Prolog only can live following a strong intuition from the programmer. On the other hand, there are isolated analysis for each paradigm. The crucial next step is to integrate in a consistent way each component of a major system.

A frame is a collection of tiled windows. Frames contains specialized windows. A frame is like a super-window, perhaps invisible, where we can put several instances of specific windows. Each window can be parameterized according to its specific task.

In the manual of XPCE, a specific notation is used for `send` and `get`: `box -> colour` sends a request to modify the colour of a box, whereas in `window <- height` we are requesting the height of a window. Because XPCE

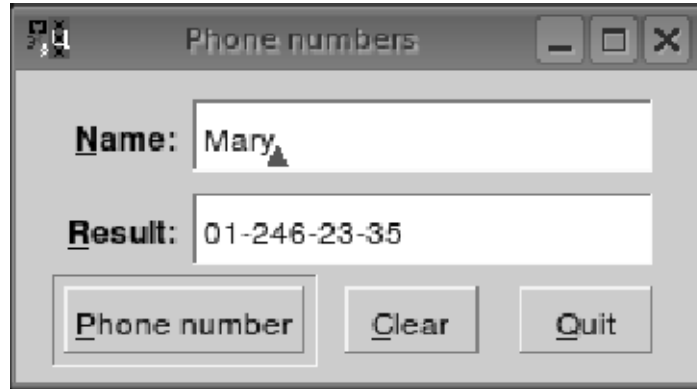


Figure 1.8: A database of names and phone numbers.

is a typed language, we can specify, to give more information about the methods of a class, the associated types in the following way:

```
box -> colour: colour
window <- height --> int
```

In the following program the Prolog machinery is combined with the XPCE input/output mechanism. This time the hard work is left to Prolog. You should note how to use the `term_to_atom` predicate in both directions: from left to right, and from right to left.

XPCE program

```
1 :-use_module(library(pce)).
2 :-op(700,xfy,=>).

3 main :-
4     new(@dialog,dialog('Term rewriting')),
5     send(@dialog,append,new(@term,
6                             text_item(term))),
7     send(@dialog,append,new(@nterm,text_item(result,''))),
8     send(@dialog,append,button(rewriting,
9                                 message(@prolog,rewrite,
10                                     @term?selection))),
11     send(@dialog,append,button(clear,
```

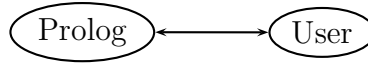


Figure 1.9: Old model of interaction in the Prolog system.



Figure 1.10: New model of interaction in the XPCE/Prolog system.

```

12         and(message(@term,clear),
13              message(@nterm,clear))),
14     send(@dialog,append,button(quit,message(@prolog,halt))),
15     send(@dialog,default_button,rewriting),
16     send(@dialog,open).

17 rewrite(Term) :-
18     term_to_atom(X,Term),
19     eq(X,NTerm),
20     term_to_atom(NTerm,NewTerm),
21     send(@nterm,value,NewTerm).

22 eq(X,Y) :- eq1(X,Y).

23 eq1(X,Z) :- eq2(X,Y),eq1(Y,Z).
24 eq1(X,X).

25 eq2(X1+Y,X2+Y) :- eq2(X1,X2).
26 eq2(X+Y1,X+Y2) :- eq2(Y1,Y2).
27 eq2(X,Y) :- X=>Y.

28 X+0 => X.
29 0+X => X.

```

```

_____ XPCE program _____
1 :- pce_global(@in_out_link, make_in_out_link).

```

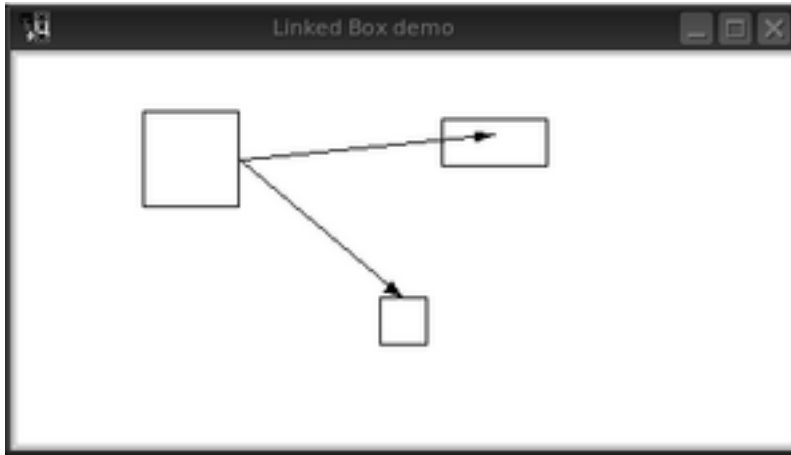


Figure 1.11: Linking graphical objects.

```

2 make_in_out_link(L) :-
3     new(L, link(in, out, line(arrows := second))).

4 main :- linked_box_demo.

5 linked_box_demo :-
6     new(P, picture('Linked Box demo')),
7     send(P, open),
8     send(P, display, new(B1, box(50,50)), point(20,20)),
9     send(P, display, new(B2, box(25,25)), point(100,100)),
10    send(P, display, new(B3, box(55,25)), point(0,100)),
11    send(B1, handle, handle(w, h/2, in)),
12    send(B2, handle, handle(w/2, 0, out)),
13    send(B3, handle, handle(w/2, h/3, out)),
14    send_list([B1, B2, B3], recogniser, new(move_gesture)),
15    send(B1, connect, B2, @in_out_link),
16    send(B1, connect, B3, @in_out_link).
17 :-main.

```

1.3 Graphical and dialogs

```
1 :-use_module(library(pce)).
2 main :-
3     new(P,window('Picture and dialog',size(200,100))),
4     new(D,dialog('Every')),
5     send(D,below,P),
6     send(P,open).
```

1.4 Declarative Input/Output in XPCE/Prolog

In the following program we show how to deal with the relationship between Prolog and XPCE:

```
1 :-use_module(library(pce)).

2 main :-
3     new(@dialog,dialog('Hello')),
4     send(@dialog,append,new(@getnumber,text_item(number))),
5     send(@dialog,append,new(@isnumber,text_item(result,''))),
6     send(@getnumber,type,int),
7     send(@dialog,append,button(result,
8         message(@prolog,fib,@getnumber?selection))),
9     send(@dialog,append,button(clear,
10         and(message(@isnumber,clear),
11             message(@getnumber,clear)))),
12     send(@dialog,append,button(quit,message(@prolog,halt))),
13     send(@dialog,open).

14 fib(N) :- fibb(N,Fib), send(@isnumber,value,Fib).

15 fibb(0,1).
16 fibb(1,1).
17 fibb(N,M) :-
```

```

18      N1 is N-1,
19      N2 is N-2,
20      fibb(N1,M1),
21      fibb(N2,M2),
22      M is M1+M2.

```

Be careful with the input part of this program: Because the implementation of fibb is very naive, the output can be delayed some time.

Leaving some code apart, we have the following schema to deal with the input/output problem:

1. Context:

```

      new(@dialog,dialog('Hello')),
      send(@dialog,append,new(@getnumber,text_item(number))),
      send(@dialog,append,new(@isnumber,text_item(result,''))),

```

2. Input:

```

      send(@dialog,append,button(result,
      message(@prolog,fib,@getnumber?selection))),

```

3. Output:

```

      fib(N) :- fibb(N,Fib), send(@isnumber,value,Fib).

```

Schematically, we can put:

1. Context:

```

      new(instances of classes): containers of information

```

2. Input:

```

      callback to hibrid-XPCE/Prolog predicates

```

3. Output:

- (a) Return of values to the context:

```
send(@isnumber,value,Fib).
```

- (b) Deduction of pure-Prolog predicates:

```
fibb(N,Fib)
```

Based on the Herbrand Universe given by `pred(Number1,Number2)` we can design a new “extended” Herbrand Universe as follows:

```
XPCE program
1 xp(
2   Number1 <User>,
3   Number2 <Ouput of Prolog predicates> ,
4   pred(Number1, Number2)
5 )
```

In particular, we would have a least Herbrand model if:

```
xp(
  Number1 <User>, and Number1 is a correct value from
  part of the user;

  Number2 <Ouput of Prolog predicates>;

  the pred(Number1, Number2) predicate is true.
)
```

Because users can have many ideas about input (detracting the designed Herbrand Universe), the best thing is enforcing to the user to select the input values from a well-designed set of values. In the previous program, this can be done as follows:

In the following program we introduce another way to input a numeric value:

XPCE program

```

1 :-use_module(library(pce)).

2 main :-
3     new(@dialog,dialog('Hello')),
4     send(@dialog,append,new(@getnumber,
5                             int_item(number,low:=0,high:=20))),
6     send(@dialog,append,new(@isnumber,text_item(result,''))),
7     send(@dialog,append,button(result,
8                                 message(@prolog,fib,@getnumber?selection))),
9     send(@dialog,append,button(clear,
10                                and(message(@isnumber,clear),
11                                       message(@getnumber,clear)))),
12     send(@dialog,append,button(quit,message(@prolog,halt))),
13     send(@dialog,open).

14 fib(N) :- fibb(N,Fib), send(@isnumber,value,Fib).

15 fibb(0,1).
16 fibb(1,1).
17 fibb(N,M) :-
18     N1 is N-1,
19     N2 is N-2,
20     fibb(N1,M1),
21     fibb(N2,M2),
22     M is M1+M2.

```

So the user is enforced to have a disciplined method to give inputs. The user have to follow a set of rules of a playing of interaction. The game is only a game: given an input, the interaction provides an output. There are not winners. It is only by the ludic pleasure of playing.

If values appear to be very restricted, in the following program we give the disciplined way to select values and also functions. This is a beautiful program dealing with several functions to be applied to an specific numeric value:

XPCE program

```

1 :-use_module(library(pce)).

2 main :-
3     new(@dialog,dialog('Hello')),
4     new(@menu,new(@menu,menu(available_functions))),
5     send(@dialog,append,new(@getnumber,
6                             int_item(number,low:=0,high:=20))),
7     send(@dialog,append,new(@isnumber,text_item(result,''))),
8     send(@dialog,append,button(result,
9                                 message(@prolog,select,
10                                         @getnumber?selection,
11                                         @menu?selection))),
12     send(@dialog,append,button(clear,
13                                 and(message(@isnumber,clear),
14                                     message(@getnumber,clear)))),
15     send_list(@menu,append,[fibonacci,factorial]),
16     send(@dialog,append,@menu),
17     send(@dialog,append,button(quit,message(@prolog,halt))),
18     send(@dialog,open).

19 select(N,fibonacci) :- fib(N).
20 select(N,factorial) :- fact(N).

21 fib(N) :- fibb(N,Fib), send(@isnumber,value,Fib).

22 fibb(0,1).
23 fibb(1,1).
24 fibb(N,M) :-
25     N1 is N-1,
26     N2 is N-2,
27     fibb(N1,M1),
28     fibb(N2,M2),
29     M is M1+M2.

30 fact(N) :- factt(N,Fact), send(@isnumber,value,Fact).

31 factt(0,1).
32 factt(N,Fact) :- N1 is N-1,

```

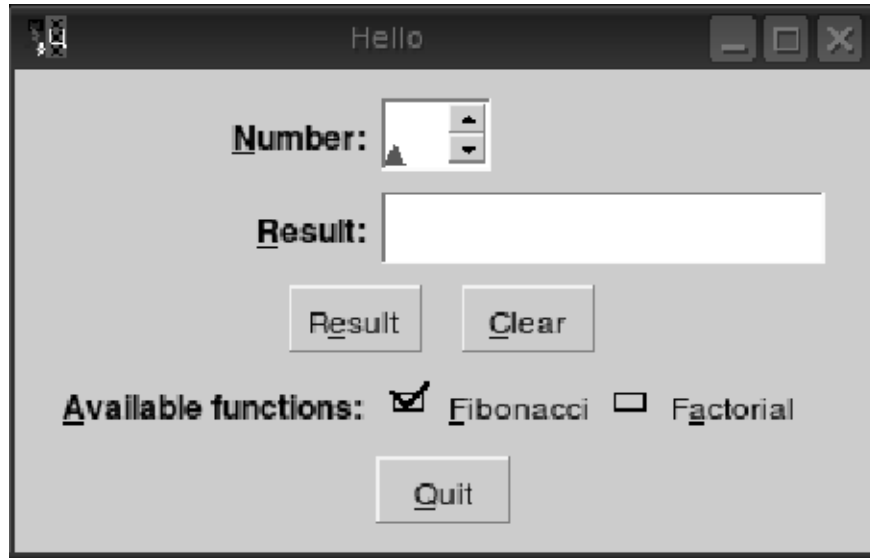


Figure 1.12: Calculating the Fibonacci function and factorial.

```

33         factt(N1,Fact1),
34         Fact is N*Fact1.

```

A screenshot of the output of this program is shown in Fig. 1.12.

Because menus are a popular way of entering selections, XPCE incorporates a system to construct them. In the following example we show how to include menus within our application:

```

X P C E   p r o g r a m
1 :-use_module(library(pce)).

2 main :-
3     new(@dialog,dialog('Hello')),
4     new(@menu,new(@menu,menu(available_functions))),
5     new(@menubar,menu_bar(menu)),

6     send(@menubar,append,new(@popup1,popup(funs))),
7     send(@menubar,append,new(@popup2,popup(values))),

```

```

8      send_list(@popup1,append,
9              [ menu_item(about,
10                  message(@display, inform,
11                      'By Manuel Hernandez'))
12                  ]),

13      send_list(@popup2,append,
14              [ menu_item(this_application,
15                  message(@display, inform,
16                      'By Manuel Hernandez')),
17                  menu_item(about_xpce,
18                      message(@display, inform,
19                          'By Jean Wielemaker'))
20                  ]),

21      send(@dialog,append,@menubar),
22      send(@dialog,append,new(@getnumber,
23                              int_item(number,low:=0,high:=20))),
24      send(@dialog,append,new(@isnumber,text_item(result,''))),
25      send_list(@menu,append,[fibonacci,factorial]),
26      send(@dialog,append,button(result,
27                              and(message(@prolog,select,
28                                  @getnumber?selection,
29                                  @menu?selection),
30                                  message(@getnumber,clear)))),
31      send(@dialog,append,@menu),
32      send(@dialog,append,button(quit,message(@prolog,halt))),
33      send(@dialog,open).

34 select(N,fibonacci) :- fib(N).
35 select(N,factorial) :- fact(N).

36 fib(N) :- fibb(N,Fib), send(@isnumber,value,Fib).

37 fibb(0,1).
38 fibb(1,1).
39 fibb(N,M) :-
40     N1 is N-1,

```

```

41      N2 is N-2,
42      fibb(N1,M1),
43      fibb(N2,M2),
44      M is M1+M2.

45 fact(N) :- factt(N,Fact), send(@isnumber,value,Fact).

46 factt(0,1).
47 factt(N,Fact) :- N1 is N-1,
48      factt(N1,Fact1),
49      Fact is N*Fact1.

```

Prolog is suitable to do symbolic processing of information. The following program gives some keys to do this kind of tasks, following the previous method of input/output:

```

_____ XPCE program _____
1 :-use_module(library(pce)).

2 main :-
3     new(@dialog,dialog('Phone numbers')),
4     send(@dialog,append,new(@name,
5                             text_item(name))),
6     send(@dialog,append,new(@isnumber,text_item(result,''))),
7     send(@dialog,append,button(phone_number,
8                                 message(@prolog,tel,
9                                         @name?selection))),
10    send(@dialog,append,button(clear,
11                                and(message(@name,clear),
12                                    message(@isnumber,clear)))),
13    send(@dialog,append,button(quit,message(@prolog,halt))),
14    send(@dialog,default_button,phone_number),
15    send(@dialog,open).

16 tel(Name) :-
17     telephone(Name,Number),
18     send(@isnumber,value,Number).

```

```

19 telephone('Karla','01-246-24-35').
20 telephone('Lety','01-246-22-35').
21 telephone('Mary','01-246-23-35').
22 telephone('Roseline','01-246-25-356').
23 telephone('Francesca','01-246-24-36').

```

To deal with nondeterminism is difficult. In this case, however, we have pragmatic way to deal with nondeterminism, by using a failure-driven cycle:

```

XPC program
1 :-use_module(library(pce)).

2 main :-
3     new(@dialog,dialog('Phone numbers')),
4     new(@brow,browser('Phone numbers')),
5     send(@dialog,append,@brow),
6     send(@dialog,append,new(@name,
7                             text_item(name))),
8     send(@dialog,append,new(@isnumber,text_item(result,''))),
9     send(@dialog,append,button(phone_number,
10                                message(@prolog,tel,
11                                       @name?selection))),
12     send(@dialog,append,button(clear,
13                                and(message(@name,clear),
14                                       message(@isnumber,clear),
15                                       message(@brow,clear)))),
16     send(@dialog,append,button(quit,message(@prolog,halt))),
17     send(@dialog,default_button,phone_number),
18     send(@dialog,open).

19 telephone('Karla','01-246-24-35').
20 telephone('Karla','01-246-26-35').
21 telephone('Karla','01-246-25-35').
22 telephone('Lety','01-246-22-35').
23 telephone('Mary','01-246-23-35').
24 telephone('Evelyne','01-246-25-356').

```

```
25 telephone('Francesca', '01-246-24-36').

26 tel(Name) :-
27     telephone(Name, Number),
28     send(@isnumber, value, Number),
29     send(@brow, append, dict_item(Number)),
30     fail.
31 tel(_).
```

1.5 Conclusions

We have taken ideas from XPCE and Prolog to reach at least two goals: First, we have presented a tutorial to use, in practice, XPCE and Prolog; second, we have also presented a method to force a declarative comprehension (at least at the high level) of side-effects in Prolog. Some future work will be intended to integrate low-level mechanism of XPCE withing Prolog. An adaptation of some concepts of monads to Logic Programming can be useful.

Mark Sergot

Jan Wielemaker

Baies

VEmden, Kowalski